

Phase 2: Concurrency & Multithreading

Week 1: Threads & Futures

Day 1 & 2: `std::thread` vs `std::jthread`

When you create a `std::thread`, you are asking the Operating System to spawn a new thread of execution.

The Danger of `std::thread`:

If a `std::thread` object goes out of scope and is destroyed before you explicitly call `.join()` (wait for it to finish) or `.detach()` (abandon it to run in the background), the C++ runtime will instantly call `std::terminate()` and crash your entire program.

The Modern Solution (`std::jthread`):

Introduced in C++20, `std::jthread` applies the RAI principles you learned in Phase 1. When a `std::jthread` goes out of scope, its destructor automatically calls `.join()`. It cleans up after itself safely.

C++

```
#include <iostream>
#include <thread>
#include <chrono>
```

```
void heavyComputation(int id) {
    std::cout << "Thread " << id << " starting work...\n";
    // Simulate work by putting the thread to sleep
    std::this_thread::sleep_for(std::chrono::milliseconds(500));
    std::cout << "Thread " << id << " finished!\n";
}
```

```
int main() {
    std::cout << "[Main] System booting up.\n";

    {
        // Spawning 3 threads using C++20 jthread
        std::jthread t1(heavyComputation, 1);
        std::jthread t2(heavyComputation, 2);
        std::jthread t3(heavyComputation, 3);

        std::cout << "[Main] I am not blocked! I can do other things while t1, t2, t3 run.\n";
    }
}
```

```

        // As we hit this closing brace, t1, t2, and t3 go out of scope.
        // Because they are jthreads, the main thread will automatically pause here
        // and wait for all three to finish. No crashes!
    }

    std::cout << "[Main] System shutting down.\n";
    return 0;
}

```

Day 3: The Hardware Reality (Context Switching)

You might think: *"If I have 10,000 user queries to process, I'll just spawn 10,000 threads!"* This will kill your performance. Your CPU only has a fixed number of physical cores. If you spawn more threads than you have cores, the OS has to rapidly pause one thread, save its memory state, load another thread's memory state, and let it run for a microsecond. This is called a **Context Switch**, and it is extremely expensive.

Always query the hardware before deciding how many threads to spawn.

```

C++

#include <iostream>
#include <thread>

int main() {
    // Returns the number of concurrent threads supported by the hardware implementation.
    unsigned int cores = std::thread::hardware_concurrency();

    std::cout << "This machine can truly run " << cores << " threads in parallel.\n";
    std::cout << "Spawning more than " << cores << " threads will cause context switching
    overhead.\n";

    return 0;
}

```

Day 4 & 5: Returning Values (std::async & std::future)

`std::thread` has a major flaw: it cannot easily return a value. If `t1` calculates a result, how does `main` get it?

Modern C++ provides `std::async`. It acts like a thread, but it returns a `std::future<T>`. A `future` is a "ticket". It represents a value that doesn't exist yet, but will exist in the future.

- `std::launch::async`: Forces the task to run on a new background thread immediately.
- `future.get()`: If you call this, and the background thread is still calculating, the main thread will block (pause) until the value is ready.

C++

```
#include <iostream>
#include <vector>
#include <numeric>
#include <future>
#include <chrono>

// A function simulating a heavy data processing task
long long processDataChunk(const std::vector<int>& data, size_t start, size_t end) {
    long long sum = 0;
    for (size_t i = start; i < end; ++i) {
        sum += data[i];
    }
    std::this_thread::sleep_for(std::chrono::milliseconds(200)); // Simulate hard work
    return sum;
}

int main() {
    std::vector<int> massiveDataset(1000000, 5); // 1 million fives
    size_t mid = massiveDataset.size() / 2;

    std::cout << "[Main] Dispatching tasks...\n";

    // Launch the first half on a background thread
    std::future<long long> futureResult = std::async(
        std::launch::async,
        processDataChunk,
        std::cref(massiveDataset), 0, mid
    );

    // Main thread processes the second half at the same time
    long long localResult = processDataChunk(massiveDataset, mid, massiveDataset.size());

    // Retrieve the background thread's result (blocks if not finished yet)
    long long backgroundResult = futureResult.get();

    std::cout << "Total Sum: " << (localResult + backgroundResult) << "\n";

    return 0;
}
```

The Weekend Assignment: "The N-Core Parallel Accelerator"

Task: Write a program that calculates the sum of a vector containing 500 million random integers (`std::vector<long long>`).

1. **Baseline:** First, write a standard `for` loop to calculate the sum on a single thread. Measure the exact time it takes using `std::chrono`.
2. **Parallelization:**
 - Query `std::thread::hardware_concurrency()` to get the number of cores (let's call it `$N$`).
 - Divide the vector into `$N$` equal chunks.
 - Use a `std::vector<std::future<long long>>` to launch `$N-1$` async tasks to process the first `$N-1$` chunks.
 - Have the main thread process the final chunk.
 - Call `.get()` on all futures, add the results together.
3. **Benchmark:** Measure the time of the parallel version. If your machine has 8 cores, your parallel code should run nearly 8 times faster than your single-threaded baseline.

Constraint: Compile using the `-O3` flag in your VSCode terminal (e.g., `g++ -O3 -std=c++20 main.cpp`) to enable maximum compiler optimizations, otherwise the debug overhead will skew your time measurements.

When you have the code and the speedup benchmark numbers ready, drop them here!

Week 2: The Chaos of Shared Memory (Data Races & Mutexes)

Last week, you learned how to spawn multiple threads to make your program run faster. But the moment two threads try to touch the exact same variable at the exact same time, your program's logic disintegrates. In backend systems, this is the #1 cause of silent data corruption.

Day 1 & 2: The Anatomy of a Data Race

In competitive programming, `count++` is one operation. To the CPU, it is **three distinct assembly instructions**:

1. **Read** the value from RAM into a CPU register.
2. **Modify** (increment) the value in the register.
3. **Write** the value back to RAM.

If Thread A and Thread B both call `count++` at the exact same microsecond, they might both **Read** the value as 0. They both **Modify** it to 1, and they both **Write** 1 back to RAM. You expected count to be 2, but it is 1.

Run this code. It *should* output 200000, but it will output a random, smaller number every time you run it.

C++

```
#include <iostream>
#include <thread>
#include <vector>

int shared_counter = 0;

void incrementWork() {
    for (int i = 0; i < 100000; ++i) {
        shared_counter++; // DATA RACE HERE
    }
}

int main() {
    std::jthread t1(incrementWork);
    std::jthread t2(incrementWork);

    // t1 and t2 will auto-join here because of jthread
    return 0;
}
```

Day 3: The Mutex and the Danger of `unlock()`

To fix a data race, we use **Mutual Exclusion** (`std::mutex`). It is a lock. If Thread A holds the lock, Thread B must go to sleep and wait until Thread A drops it.

The CP / Novice Way (Dangerous):

C++

```
std::mutex mtx;
mtx.lock();
shared_counter++;
// IF AN EXCEPTION IS THROWN HERE, mtx.unlock() IS NEVER CALLED.
// EVERY OTHER THREAD WILL WAIT FOREVER. (DEADLOCK)
mtx.unlock();
```

The Systems Engineer Way (`std::lock_guard`):

We use RAII (from Phase 1). `std::lock_guard` locks the mutex in its constructor and *guarantees* it will unlock in its destructor when it goes out of scope, even if an exception is thrown.

C++

```
#include <iostream>
#include <thread>
#include <vector>
#include <mutex>

int shared_counter = 0;
std::mutex mtx; // Protects shared_counter

void incrementWork() {
    for (int i = 0; i < 100000; ++i) {
        // The lock is acquired here
        std::lock_guard<std::mutex> lock(mtx);
        shared_counter++;
        // The lock is automatically released here at the end of the scope loop
    }
}

int main() {
    {
        std::jthread t1(incrementWork);
        std::jthread t2(incrementWork);
    } // Wait for threads to finish

    std::cout << "Final Count: " << shared_counter << "\n"; // Will ALWAYS be 200000
    return 0;
}
```

Day 4: The Deadlock Trap

A Deadlock happens when two threads are waiting for each other forever.

Imagine a bank transfer system protecting two accounts.

- **Thread A (Transfer \$10 from Acc1 to Acc2):** Locks Acc1's mutex. Needs to lock Acc2's mutex.
- **Thread B (Transfer \$20 from Acc2 to Acc1):** Locks Acc2's mutex. Needs to lock Acc1's mutex.

Both threads are holding one lock and waiting for the other. The server completely freezes.

Day 5: Modern Locking (`std::scoped_lock`)

Introduced in C++17, `std::scoped_lock` uses a special algorithm (often the dining philosophers algorithm) to lock multiple mutexes at the exact same time without ever causing a deadlock, regardless of the order they are passed in.

C++

```
#include <iostream>
#include <mutex>
#include <thread>
#include <functional>

class BankAccount {
public:
    int balance;
    std::mutex mtx;
    BankAccount(int b) : balance(b) {}
};

void safeTransfer(BankAccount& from, BankAccount& to, int amount) {
    // Locks both mutexes simultaneously and safely. No deadlocks!
    std::scoped_lock lock(from.mtx, to.mtx);

    from.balance -= amount;
    to.balance += amount;
    std::cout << "Transfer complete.\n";
}

int main() {
    BankAccount acc1(100);
    BankAccount acc2(100);

    std::jthread t1(safeTransfer, std::ref(acc1), std::ref(acc2), 10);
    std::jthread t2(safeTransfer, std::ref(acc2), std::ref(acc1), 20);

    return 0;
}
```

The Weekend Assignment: "The Thread-Safe LRU Cache"

Since you are a CP expert, you likely know how to build an LRU (Least Recently Used) Cache using a `std::list` and a `std::unordered_map` for $O(1)$ gets and puts.

Your Task:

Take the standard LeetCode 146 (LRU Cache) problem and make it **production-ready and thread-safe**.

1. Write a `ThreadSafeLRUCache` class.
2. It must have multiple worker threads concurrently trying to `put()` and `get()` random keys.
3. You must protect the internal `std::list` and `std::unordered_map` with a `std::mutex`.
4. **The Bottleneck Challenge:** If you lock the *entire* cache every time someone reads from it, your multithreaded cache will actually be slower than a single-threaded one (this is called lock contention). Think about how you might optimize this, or at least measure the performance drop compared to a completely unlocked map.

Thread Safe LRU Implementation using the RULE of 4 ½ and smart pointers

```
#include <memory>
#include <unordered_map>
#include <iostream>
#include <mutex>
#include <functional>

/**
 * Motive : To practice Modern C++ conventions and Deoxygen styled Comments
 * @file LRU_with_thread_safety.cpp
 * @author Aniket Kundu
 * @brief A thread-safe LRU (Least Recently Used) Cache Implementation
 * @version 0.1
 * @date 2026-03-28
 */

template<typename T>
class Node{
public:
    int key;
    T val;
    Node<T> *prev, *next;
    Node(int k , T v): key(k), val(v), prev(nullptr), next(nullptr) {}
};
```



```

};

/**
 * @brief A Least Recently Used (LRU) Cache.
 * Stores key-value pairs with a fixed capacity.
 * When capacity is reached,
 * the least recently accessed item is evicted.
 * @tparam T The type of data to be stored in the cache.
 * @note Time complexity: Get and Put operations are O(1) average.
 */
template<typename T>
class LRU{
private:
    int capacity;
    std::unordered_map<int, std::unique_ptr<Node<T>>> cache;
    std::unique_ptr<Node<T>> head, tail;

    mutable std::mutex mtx;

    void removeNode(Node<T>* node) {                // remove the node from the
Linked list
        node->prev->next = node->next;
        node->next->prev = node->prev;
    }
    void addToTail(Node<T>* node) {                // add just before tail in
Linked List
        node->next = tail.get();
        node->prev= tail->prev;
        tail->prev->next = node;
        tail->prev = node;
    }
public:
    LRU(int cap) :
        capacity(cap)
    {
        // default constructor
        tail = std::make_unique<Node<T>>(0, T());
    }

```

```

    head = std::make_unique<Node<T>>(0, T());
    tail->prev = head.get();
    head->next = tail.get();
}
// no destructor is needed as unique_ptr is used

LRU(const LRU<T>& other):
capacity(other.capacity)
{
    // copy constructor
    tail = std::make_unique<Node<T>>(0, T());
    head = std::make_unique<Node<T>>(0, T());

    head->next = tail.get();
    tail->prev = head.get();

    // creation lock
    std::lock_guard<std::mutex> lock(other.mtx);
    Node<T>* cur = other.head->next;
    while(cur != other.tail.get()){
        this->put(cur->key, cur->val);
        cur = cur->next;
    }
    // creation unlock
}

LRU(LRU<T>&& other) noexcept : capacity(0)
{
    // move constructor
    // nullify the input object raw pointers
    std::lock_guard<std::mutex> lock(other.mtx);
    tail = std::make_unique<Node<T>>(0, T());
    head = std::make_unique<Node<T>>(0, T());

    head->next = tail.get();
    tail->prev = head.get();

    swap(other);
    // creation unlock
}

```

```

}

/**
 * @brief Unified Assignment Operator
 * though we pass by value, the compiler first creates a local instance
for "other"
 * just prior to entry into the function. If rvalue is involved,
 * then the compiler automatically makes a call to the move constructor
 * , same goes for lvalue - copy constructor is called
 * @note This is the golden standard followed by industry pros
 * no longer the need for manual garbage value handling, and exception
safe
 */
LRU& operator=(LRU<T> other){ // smart compiler targets the respective
constructors
    // creation lock
    std::lock_guard<std::mutex> lock(mtx);
    this->swap(other);
    return *this;
    // creation unlock
}

/**
 * @brief a helper for swapping the values in case of copy and move
 */
void swap(LRU<T>& other) noexcept {
    std::swap(capacity, other.capacity);
    std::swap(cache, other.cache);
    std::swap(head, other.head);
    std::swap(tail, other.tail);
}

/**
 * @brief Puts a key-value pair into the cache
 * If the key exists, the value is updated,
 * and the entry is brought to the "Most Recently Used" (MRU) position.
 * If the key doesnt exist, a new key-value pair is made and inserted

```

```

    * at the MRU position,
    * and if capacity is being exceeded then the LRU element is removed
    * to accomodate the incoming new key-value pair
    * @param key The unique identifier for the cached item.
    * @param val The value of the cached item.
    * @return void
    */
void put(int key, T val){
    std::lock_guard<std::mutex> lock(mtx);
    if(cache.find(key) != cache.end()){
        Node<T>* node = cache[key].get();
        node->val = val;
        removeNode(node);
        addToTail(node);
    }else{
        //lock
        if(capacity <= cache.size()){ // remove the least
recently used if capacity is exceeded
            Node<T>* lru = head->next;
            removeNode(lru); // removes the lru from linked list
            cache.erase(lru->key); // automatically deletes lru, as
unique_ptr
        }
        auto newnode = std::make_unique<Node<T>>(key, val);
        Node<T>* rawptr = newnode.get(); // always secure the raw pointer
separate using get()
        cache[key] = std::move(newnode);
        addToTail(rawptr); // add the raw pointer to linked list
    }
}

/**
 * @brief Retrieves a value from the cache.
 * If the key exists, it is moved to the "Most Recently Used" position.
 * @param key The unique identifier for the cached item.
 * @return T The cached value, or a default-constructed T if not found.

```

```

    */
    T get(int key) {
        // lock
        std::lock_guard<std::mutex> lock(mtx);
        if(cache.find(key) != cache.end()){
            auto node = cache[key].get();
            removeNode(node);
            addToTail(node);
            return node->val;
        }else{
            return T();
        }
        // unlock
    }

    void display(){
        // lock
        std::lock_guard<std::mutex> lock(mtx);
        Node<T>* cur = head->next;
        while(cur != tail.get()){
            std::cout << "[" << cur->key << ", " << cur->val << "]" \n";
            cur = cur->next;
        }
        std::cout << std::endl;
        // unlock
    }
};

int main() {
    LRU<std::string> obj(3);
    obj.put(1, "First Entry");
    obj.put(2, "Second Entry");
    obj.put(3, "Third Entry");
    obj.display();
    std::cout << obj.get(2) << std::endl;
}

```

```

obj.display();
obj.put(4, "Fourth Entry");
obj.display();
std::cout << obj.get(3) << std::endl;
obj.display();
}

```

Week 3: Thread Coordination (Condition Variables).

Last week, you learned how to use a `std::mutex` to stop threads from destroying shared data. But a mutex only provides *protection*. It does not provide *communication*.

In real-world backend systems, threads need to talk to each other. If Thread A is downloading an image, and Thread B is supposed to run an ML inference on that image, Thread B needs to know exactly when Thread A finishes downloading it.

Day 1 & 2: The Inefficiency of "Polling"

The instinct of someone moving from single-threaded algorithms to multi-threading is to use a `while` loop to check if data is ready. This is called **polling**, and it is toxic to system performance.

C++

```

// BAD SYSTEMS ENGINEERING (POLLING)
bool data_ready = false;

void consumerThread() {
    // This loop spins millions of times per second.
    // It will consume 100% of a CPU core doing absolutely nothing.
    while (!data_ready) {
        // Do nothing, just keep checking
    }
    std::cout << "Data is finally ready!\n";
}

```

If you deploy this on an AWS EC2 instance, your CPU usage will spike to 100%, your server will overheat, and you will be billed for wasted compute cycles.

Instead of polling, the consumer thread needs to **go to sleep** (yielding the CPU back to the OS) and ask to be **woken up** only when the data is ready.

Day 3: `std::condition_variable` & `std::unique_lock`

A Condition Variable (`std::condition_variable`) allows a thread to suspend execution until another thread notifies it.

To use it, you must pair it with a `std::unique_lock`.

Why not `std::lock_guard`? Because when a thread goes to sleep, it *must* release the mutex so the other thread can acquire it to produce the data. `std::lock_guard` cannot be unlocked temporarily. `std::unique_lock` can.

Here is the precise sequence of events when `cv.wait()` is called:

1. It atomically **unlocks** the mutex and puts the thread to **sleep**.
2. ... (Thread is asleep, consuming 0% CPU) ...
3. Another thread calls `cv.notify_one()`.
4. The sleeping thread wakes up and immediately **re-locks** the mutex before continuing.

Day 4: The "Spurious Wakeup" Trap

Operating Systems are chaotic. Sometimes, the OS will wake up a sleeping thread even if nobody called `notify()`. This is known as a **Spurious Wakeup**.

If your thread wakes up randomly and assumes the data is ready, it will process garbage memory and crash. To fix this, `cv.wait()` takes a **lambda predicate** as a second argument. It acts as a double-check.

C++

```
// The correct, safe way to wait
```

```
cv.wait(lock, []{ return data_ready == true; });
```

Under the hood, this simply translates to:

C++

```
while (!data_ready) {
```

```
cv.wait(lock); // If woken up spuriously, the while loop checks the condition again!
```

```
}
```

Day 5: The Producer-Consumer Pattern

This is the most famous architectural pattern in backend engineering. You have one or more "Producers" creating tasks (e.g., handling incoming network requests) and one or more "Consumers" processing them (e.g., executing the Heavy C++ Inference).

Here is a perfectly synchronized Producer-Consumer in Modern C++:

C++

```
#include <iostream>
#include <thread>
#include <mutex>
#include <condition_variable>
#include <queue>
#include <chrono>

std::queue<int> task_queue;
std::mutex mtx;
std::condition_variable cv;
bool production_finished = false;

void producer() {
    for (int i = 1; i <= 5; ++i) {
        std::this_thread::sleep_for(std::chrono::milliseconds(200)); // Simulate work

        {
            std::lock_guard<std::mutex> lock(mtx);
            task_queue.push(i);
            std::cout << "[Producer] Generated task " << i << "\n";
        }

        // Notify ONE sleeping consumer that a task is available
        cv.notify_one();
    }

    {
        std::lock_guard<std::mutex> lock(mtx);
        production_finished = true;
    }
}
```



```

    cv.notify_all(); // Wake up any remaining consumers so they can exit
}

void consumer(int id) {
    while (true) {
        std::unique_lock<std::mutex> lock(mtx);

        // Go to sleep UNTIL the queue is not empty OR production is finished
        cv.wait(lock, []{ return !task_queue.empty() || production_finished; });

        // If the queue is empty and production is done, exit the thread
        if (task_queue.empty() && production_finished) {
            std::cout << "[Consumer " << id << "] Shutting down.\n";
            break;
        }

        // Grab the task
        int task = task_queue.front();
        task_queue.pop();

        // Unlock manually before processing so other consumers can grab tasks!
        lock.unlock();

        std::cout << "[Consumer " << id << "] Processing task " << task << "... \n";
        std::this_thread::sleep_for(std::chrono::milliseconds(300)); // Simulate processing
    }
}

int main() {
    std::cout << "--- Starting Producer-Consumer Engine ---\n";

    std::jthread p1(producer);
    std::jthread c1(consumer, 1);
    std::jthread c2(consumer, 2);

    return 0;
}

```

The Weekend Assignment: "The Thread-Safe Blocking Queue"

To build a professional Thread Pool (which is required for high-performance servers like Redis), you need a robust, reusable data structure that encapsulates all this messy lock/notify logic.

Your Task:

Create a generic `BlockingQueue<T>` class template.

Requirements:

1. It wraps a standard `std::queue<T>`.
2. It contains its own `std::mutex` and `std::condition_variable`.
3. **`void push(T item)`**: Locks the mutex, pushes the item via move semantics (`std::move`), and calls `notify_one()`.
4. **`std::optional<T> pop()`**: Locks the mutex, waits on the condition variable until the queue is not empty or an "abort" flag is set. Returns the item, or `std::nullopt` if the queue was aborted.
5. **`void abort()`**: Sets a flag and calls `notify_all()` to wake up any threads stuck waiting in `pop()`, allowing them to safely exit.

Once you build this `BlockingQueue`, you have essentially completed 80% of the architecture required to build a custom C++ Thread Pool.

Do you want to write the `BlockingQueue` class yourself, or shall we move directly to Week 4 where we use it to build the actual **Thread Pool Engine**?

Thread pool Architecture and Idea:

Creating and destroying a thread is actually a highly expensive operation for your computer's operating system.

When you ask the OS to create a new thread, it has to allocate memory for the thread's stack, set up execution context, and register it with the CPU scheduler. If you do this hundreds of times a second in a program, that overhead will completely wipe out any speed boost you gained from multithreading.

To solve this, we use a design pattern called a **Thread Pool**.

How a Thread Pool Works

Instead of creating a thread for a specific task and destroying it when the task is done, you:

1. Create a fixed number of "worker" threads when your program starts.
2. Put these threads into an infinite loop where they sleep/wait.
3. When you have work to do, you throw a "Task" into a shared **Queue**.
4. One of the sleeping worker threads wakes up, grabs the task from the queue, executes it, and then goes right back to waiting for the next task.

```

class ThreadPool {
private:
    std::vector<std::thread> workers;           // The actual threads
    std::queue<std::function<void()>> tasks;    // The queue of tasks

    std::mutex queue_mutex;                    // Protects the queue
    std::condition_variable condition;         // Wakes up sleeping threads
    bool stop = false;                         // Flag to shut down the pool

public:
    // Constructor: Spin up 'N' threads
    ThreadPool(size_t num_threads) {
        for(size_t i = 0; i < num_threads; ++i) {
            workers.emplace_back([this] {
                while(true) {
                    std::function<void()> task;

                    // Scope block for the mutex lock
                    {
                        std::unique_lock<std::mutex> lock(this->queue_mutex);

                        // Wait until there is a task OR the pool is stopping
                        this->condition.wait(lock, [this] {
                            return this->stop || !this->tasks.empty();
                        });

                        // If stopping and no tasks left, exit the thread loop
                        if(this->stop && this->tasks.empty()) {
                            return;
                        }

                        // Grab the task
                        task = std::move(this->tasks.front());
                        this->tasks.pop();
                    }
                    // Lock is released here

                    // Execute the task outside the lock so other threads aren't blocked
                    task();
                }
            });
        }

        // Add a new task to the queue
        void enqueue(std::function<void()> task) {
            {
                std::unique_lock<std::mutex> lock(queue_mutex);
                tasks.push(std::move(task));
            }
            condition.notify_one(); // Wake up one sleeping worker thread
        }
    }
}

```

```

// Destructor: Clean up threads when the pool is destroyed
~ThreadPool() {
    {
        std::unique_lock<std::mutex> lock(queue_mutex);
        stop = true;
    }
    condition.notify_all(); // Wake up ALL threads so they can finish and exit

    for(std::thread &worker: workers) {
        worker.join(); // Wait for all threads to finish
    }
}

};

// --- Example Usage ---
int main() {
    std::cout << "Starting Thread Pool with 4 threads...\n";
    ThreadPool pool(4);

    // Throw 10 tasks at the pool
    for(int i = 1; i <= 10; ++i) {
        pool.enqueue([i] {
            std::cout << "Task " << i << " is executing on thread "
                << std::this_thread::get_id() << "\n";

            // Simulate some work taking 1 second
            std::this_thread::sleep_for(std::chrono::seconds(1));
        });
    }

    std::cout << "All tasks enqueued. Main thread waiting...\n";

    // Give the pool time to finish before main() exits
    std::this_thread::sleep_for(std::chrono::seconds(4));
    std::cout << "Done.\n";

    return 0;
}

```

Why this code is fast (The Key Concepts)

- **No Spawning Overhead:** The `std::thread` objects are only created *once* in the constructor. If you enqueue 10,000 tasks, you are still only using those original 4 threads.

- **std::condition_variable:** This is the secret sauce. Instead of having threads constantly checking the queue in a high-CPU loop (called "busy waiting"), `condition.wait()` puts the threads to sleep at the OS level. They consume **zero CPU cycles** until `condition.notify_one()` wakes them up.
- **Scoped Locking:** We use a `std::mutex` to ensure two threads don't try to grab the same task at the exact same millisecond, which would crash the program. Notice how `task();` is called *outside* the `{ }` block where the lock lives. This allows threads to run their tasks concurrently.

Would you like me to explain how to modify this so your tasks can return values (like returning the result of a math calculation) back to the main thread?

std::atomic

While mutexes and condition variables are the standard tools for synchronization, they come with a heavy performance cost: **context switching**. When a thread hits a locked `std::mutex`, the Operating System steps in, puts the thread to sleep, and wakes it up later. That OS intervention takes microseconds—an eternity in low-latency systems.

`std::atomic` allows you to bypass the Operating System entirely.

1. What is an Atomic Operation?

An atomic operation is indivisible. When you declare a variable as `std::atomic<int> counter;`, the compiler translates operations on this variable into specialized, lock-free hardware instructions baked directly into the CPU silicon.

If Thread A and Thread B simultaneously call `counter++`, the hardware guarantees that neither thread will read a half-written value, and neither thread will be put to sleep.

2. The Core Mechanic: Compare-and-Swap (CAS)

While `counter++` is easy, complex lock-free data structures (like a lock-free queue or stack) rely on a foundational algorithm called **Compare-and-Swap (CAS)**. In C++, this is implemented as `compare_exchange_weak` or `compare_exchange_strong`.

Instead of locking data, computing, and writing, a thread using CAS does this:

1. **Read** the current shared value into a local variable.
2. **Compute** the new desired value locally.
3. **CAS Instruction:** Tell the CPU, *"Swap the shared value with my new value, BUT ONLY IF the shared value hasn't been changed by someone else since I read it."*
4. If it was changed by another thread, the swap fails, and the thread instantly loops back to Step 1 to try again.

Here is what a lock-free increment looks like in C++ using CAS:

C++

```
#include <atomic>
```

```
std::atomic<int> shared_val{0};
```

```
void lock_free_increment() {  
    int expected = shared_val.load(); // Step 1: Read  
    int desired;  
  
    do {  
        desired = expected + 1;    // Step 2: Compute  
        // Step 3: CAS  
        // If shared_val == expected, it sets shared_val = desired and returns true.  
        // If shared_val != expected, it updates 'expected' with the new shared_val and returns false.  
    } while (!shared_val.compare_exchange_weak(expected, desired));  
}
```

3. Memory Ordering (Release-Acquire)

When you use `compare_exchange_weak`, the compiler and the CPU are allowed to reorder your instructions for optimization, as long as it doesn't break the logic *within that single thread*. However, in multi-threaded programming, this reordering can cause disaster.

Memory Ordering tells the compiler how strictly to enforce the sequence of instructions across different threads.

- `std::memory_order_relaxed`: Do whatever is fastest. Only guarantee the atomic operation itself.
- `std::memory_order_release` / `std::memory_order_acquire`: Ensures that if Thread A "releases" a flag, Thread B "acquires" it and is guaranteed to see all memory writes Thread A did *before* the release. This is how you safely publish data between threads without locks.

Lock-free programming is incredibly fast but notoriously difficult to debug because bugs often involve hardware-level memory reordering.

Treiber Stack

A lock-free stack (Treiber stack) is essentially a singly linked list where the `head` pointer is an atomic variable. Multiple threads will try to update the `head` pointer simultaneously using Compare-And-Swap (CAS).

Here is the basic structure. Notice there is not a single `std::mutex` in this entire class.

C++

```
#include <atomic>
```

```
#include <memory>
```

```
template<typename T>
```

```
class LockFreeStack {
```

```
private:
```

```
    struct Node {
```

```
        T data;
```

```
        Node* next;
```

```
        Node(const T& data) : data(data), next(nullptr) {}
```

```
    };
```

```
    // The ONLY shared state.
```

```
    // std::atomic ensures that reads/writes to this pointer are thread-safe.
```

```
    std::atomic<Node*> head{nullptr};
```

```
public:
```

```
    void push(const T& data) {
```

```
        Node* new_node = new Node(data);
```

```
        // Step 1: Read current head
```

```
        new_node->next = head.load(std::memory_order_relaxed);
```

```
        // Step 2 & 3: Compute and CAS
```

```
        // "Swap 'head' with 'new_node', but ONLY IF 'head' is still equal to 'new_node->next'."
```

```
        // If it fails, CAS automatically updates 'new_node->next' with the fresh 'head' value.
```

```
        while (!head.compare_exchange_weak(new_node->next, new_node,
```

```
            std::memory_order_release,
```

```
            std::memory_order_relaxed)) {
```

```
            // The loop body is empty! The CPU just spins incredibly fast until it wins the race.
```

```
}  
}
```

```
bool pop(T& result) {  
    // Step 1: Read current head  
    Node* old_head = head.load(std::memory_order_relaxed);  
  
    // Step 2 & 3: Compute and CAS  
    // "Swap 'head' with 'old_head->next', but ONLY IF 'head' is still equal to 'old_head'."  
    while (old_head && !head.compare_exchange_weak(old_head, old_head->next,  
                                                    std::memory_order_acquire,  
                                                    std::memory_order_relaxed)) {  
        // Spin until we successfully point head to the next node.  
    }  
  
    // If the stack was empty  
    if (!old_head) {  
        return false;  
    }  
  
    // Extract the data  
    result = old_head->data;  
  
    // --- DANGER ZONE ---  
    // In a strictly basic implementation, we delete the old node here:  
    // delete old_head;  
    // -----  
  
    return true;  
}
```



```
};
```

The System Design Interview Trap: The ABA Problem

If you write the code exactly as above in an interview, the interviewer will ask: *"What happens if you run `delete old_head`; at the end of `pop()`?"*

If you delete it blindly, your system will eventually crash. Here is why lock-free programming is considered "dark magic":

1. **Thread 1** calls `pop()`. It reads `old_head = Node A`. It gets paused by the OS right before the CAS instruction.
2. **Thread 2** swoops in, calls `pop()`, successfully removes Node A, and `deletes` it.
3. **Thread 2** pushes a completely new Node, but the memory allocator recycles the exact same memory address that Node A used! Let's call this new node Node A'.
4. **Thread 1** wakes up. It executes its CAS: *"Is the head still pointing to the address of Node A?"* 5. The CPU looks at the memory address, sees that it matches (because Node A' is sitting at the exact same hex address), and says *"Yes!"* Thread 1 successfully executes the CAS, but the `old_head->next` pointer it had saved is now pointing to garbage or corrupted memory because the original list structure was destroyed. This is the notorious **ABA Problem** (A changed to B, then back to A).

How do systems engineers solve this?

In high-performance C++, you do not delete nodes immediately in a lock-free data structure. You use memory reclamation techniques like **Hazard Pointers** or **Epoch-Based Reclamation** (EBR) to ensure a node is only deleted once *all* threads are guaranteed to be done looking at it.

Those are the deepest end of the C++ systems pool. You have moved past writing the logic of a data structure and are now dealing with the raw lifecycle of memory in a highly concurrent environment.

In a lock-free structure, when Thread A calls `pop()` and removes a node from the linked list, it **cannot** immediately call `delete`. If Thread B is paused by the OS exactly at the moment it was looking at that same node's memory address, and then wakes up, Thread B will read freed memory (a segfault) or corrupted recycled memory (the ABA problem).

To solve this, we decouple **unlinking** (removing the node from the stack) from **reclaiming** (calling `delete`). We only reclaim memory when we can mathematically guarantee no thread is looking at it anymore.

Here are the two dominant architectures for solving this.

1. Hazard Pointers (The "Bookmark" Approach)

Hazard pointers are a highly granular, per-thread protection mechanism. Think of it like reading a book in a library. If you put a bookmark (hazard pointer) on a page, the librarian (the garbage collector) knows not to throw that page away, even if the book is being discarded.

How it works:

1. **The Announcement:** Before Thread A reads a node, it writes the memory address of that node into a globally visible array called the "Hazard Array". This is Thread A saying: *"I am currently looking at memory address 0x1A2B. Do not delete it."*
2. **The Verification:** Thread A then double-checks that the node hasn't been removed by another thread while it was writing to the Hazard Array.
3. **The Delayed Deletion:** Thread B comes along and successfully unlinks the node at 0x1A2B from the stack. Thread B wants to delete it.
4. **The Check:** Thread B scans the global Hazard Array. It sees Thread A's bookmark on 0x1A2B. Instead of deleting it, Thread B pushes the node into a local "Retire List" (a to-be-deleted-later queue).
5. **The Cleanup:** Periodically, threads will scan their Retire Lists against the Hazard Array. If the bookmark is gone, the node is safely deleted.

The Trade-off: Hazard pointers are highly memory-efficient because they clean up garbage quickly. However, writing to the shared Hazard Array on every single read requires a strict memory barrier (`std::memory_order_seq_cst`), which slows down the CPU's execution pipeline.

2. Epoch-Based Reclamation (EBR) (The "Generational" Approach)

Because Hazard Pointers are relatively slow, top-tier systems engineering libraries (like Facebook's Folly) usually prefer Epoch-Based Reclamation. Instead of tracking individual nodes, EBR tracks **time phases** (Epochs).

Think of it like restaurant shifts. You cannot throw away the order tickets from the morning shift until you are absolutely certain every single cook from the morning shift has clocked out.

How it works:

1. **The Epoch Counter:** There is a globally shared atomic integer called the `Global Epoch` (e.g., $E = 100$).
2. **Entering the Arena:** When Thread A wants to do *any* lock-free operation (push/pop), it reads the Global Epoch and sets its own local state: *"Thread A is now active in Epoch 100."*
3. **The Retirement Bin:** When Thread B successfully unlinks a node, it does not check any hazard arrays. It simply throws the node into a garbage bin explicitly labeled "Retired in Epoch 100".

4. **Advancing Time:** The Global Epoch can only advance to 101 when **all** active threads have caught up to Epoch 100.
5. **The Safe Deletion Rule:** You can safely delete all the nodes sitting in the Epoch E garbage bin only when the Global Epoch reaches $E + 2$.
 - *Why + 2?* If the Global Epoch has reached 102, it mathematically guarantees that every single thread has finished whatever it was doing in Epoch 100. The old nodes are completely invisible to the current threads.

The Trade-off: EBR is incredibly fast because threads only have to read a global integer instead of writing to a shared array. However, it has one fatal flaw: **The Stalled Thread Problem**. If Thread A enters Epoch 100, and then the Operating System puts Thread A to sleep for 5 seconds, the Global Epoch cannot advance. Meanwhile, Thread B keeps popping thousands of nodes, putting them in the Retire List. The system cannot delete *anything* until Thread A wakes up, causing a massive temporary memory leak.

When building your high-throughput C++ systems, unless you are writing ultra-low latency HFT code where every nanosecond counts, standard `std::shared_mutex` (like you used in your distributed lock manager) is often safer and easier to maintain. But knowing how EBR works is what separates a standard developer from a principal engineer.